

UNIVERSITY OF BIELEFELD

BACHELOR THESIS

Context Graphs: Reconstructable Language Models Based on Sub-String Relation Graphs

Author: Linus Behrbohm

Supervisors: Dr. Basil Ell

Prof. Dr. Philipp Cimiano

August 27, 2026

Abstract

Recent advances in natural language modelling, driven by the transformer model, have drastically increased expressive power and raised new awareness to standing challenges and risks associated with employing artificial intelligence at a large scale. Although existing language models exhibit convincing generative abilities, they are generally regarded as black-boxes, are known to generate convincing false information and are difficult to update in a directed manner. In this paper we introduce the context graph model, a natural language model based on the reduced sub-string relation for a given raw text corpus. We build the formal foundation for this model by providing a mathematical definition of the model and describe an algorithm for inducing the context graph structure from raw text examples. With this research we present a concept to induce a reconstructable, updatable and human-readable language model from a raw text corpus.

CONTENTS

	Abstract	
1	Introduction	1
	1.1 Reliability	1
	1.2 Interpretability & Explainability	2
	1.3 Updatability	3
2	Related Concepts	4
	2.1 Language Modelling	4
	2.2 Distributional Semantics	6
	2.3 Attention Mechanism	6
	2.4 Grammar Induction	7
3	Model Definition	8
	3.1 Context Representation	8
	3.2 Reducing the Number of Edges	9
	3.3 Reducing the Number of Nodes	10
	3.4 Maximum number of nodes over an alphabet	12
	3.5 Reconstructing Training Data	14
	3.6 Counting n-gram Frequencies	15
4	Induction Algorithm	17
	4.1 Search	17

4.2	Insert	19
4.2.1	<i>Splitting Nodes</i>	20
4.2.2	<i>Removing Repetitions</i>	22
4.2.3	<i>Joining Partitions</i>	23
4.3	Read	25
4.3.1	<i>Recording overlapping Nodes</i>	25
4.3.2	<i>Continuously Updating the Grammar</i>	27
4.3.3	<i>Update Step</i>	28
5	Discussion	29
5.1	Predicting Context by Reconstruction.	29
5.2	Smoothing Techniques	31
5.2.1	<i>Syntactic Similarity</i>	31
5.2.2	<i>Semantic Similarity</i>	32
5.3	Accepting Novel Inputs	33
6	Conclusion	34

1 Introduction

Early generative language models, such as the ELIZA system [1], were built using hand-crafted rules derived from linguistic research and domain knowledge. Compared to modern language models (LM), these programmatic systems were very expensive to develop and still limited in their comprehensive and generative capabilities. Today, research has increasingly turned towards probabilistic, data-driven systems which are more robust to unseen inputs and can often be trained automatically on massive datasets even without human supervision. The most advanced LM architectures are deep neural networks trained on gigantic data sets, such as the content of the World Wide Web. By optimizing increasingly larger real-valued statistical models, and fine-tuning these general models to more specific tasks, we have been able to continuously improve their linguistic capabilities [2].

Despite the impressive performances of these models, many challenges remain with the transparency and reliability of these models. Research has pointed out logical inconsistency of generated texts [3], reliably reproducing factual knowledge from the training datasets [4] and with the transparency of the decisions made by large LMs (LLM) [5]. Many potential use-cases for LMs such as advisor systems, search engines or command interpreters require reliable solutions to these problems.

Additionally, neural LMs are generally found to be difficult to update and adjust to local changes in the training data [6]. LLMs require to be retrained on the updated training data set to reliably perform the update. Because the models are hard to interpret it is difficult to predict how to update the model parameters directly to change or remove specific parts of its original training data set.

Rule based systems may address these issues effectively, however so far completely rule-based models are still very expensive to develop as they are largely hand-crafted and unable to capture all the complexity of large natural language data sets.

We will focus on three properties of language models: **interpretability**, **updatability** and **reliability**. In the remaining study we define a rule based language model effectively satisfying these properties. We then describe an algorithm to automatically induce the model from an unsupervised dataset of raw sequence data. Finally, we explore applications and additional algorithms for extracting knowledge from the induced structure.

1.1. Reliability

Language models should not only produce grammatically correct results, they also have to understand the semantic meaning and respect factual knowledge presented in the training data. Although deep neural LMs produce intuitively convincing texts which are very similar to the texts from its training, they can often make claims inconsistent with factual knowledge or even basic logic, which presents

a problem for many applications which rely on the model being truthful and reliable. What makes matters worse is that these models can deliver answers confidently while making false or inconsistent statements.

“Hallucinations” in deep neural language models have been investigated and related to the more desirable “creativity” as an expression of the same circumstance, where the model predicts a text with a low probability [7], due to the input not being extensively represented in training, as in few or zero-shot learning. This does allow for novel generations, which may be desirable, but there is a need to reliably detect a lack of knowledge in a trained model. For neural models, this is challenging, evident from the inability to reliably **reconstruct** their training data from the trained model [8]. This means deep neural models are inherently prone to generate text that is not represented in their training dataset, and they will fabricate responses even when not been trained on the inputs, creating a risk of making false claims or misrepresentations of reality.

1.2. Interpretability & Explainability

When we use a language model, we not only expect it to be trustworthy, we also want to understand how model arrives at its answers. A language model is interpretable when a human user can understand how the model’s internal structure is derived from its training data. It is explainable when they can follow the model’s reasoning from an input prompt to the generated output.

The internal structure of a deep neural network is not generally interpretable, because the original training data can not be reconstructed. Therefore, it is also extremely difficult to explain the model’s responses in terms of its training data. If we want to rely on natural language models for knowledge extraction or decision-making, we need some way to verify the sources the responses are based upon. There have been attempts to expand neural LMs with database retrievals in Retrieval Augmented Generation [9], however these methods are still subject to inaccuracies in their non-interpretable parametric models.

Although we would often like to use language models as classical knowledge bases, it has been established that neural networks are not knowledge bases, but “statistical models of knowledge bases” [10]. This is advantageous for creative tasks where errors or fabricated information is not a problem, but it disqualifies these models from automated knowledge extraction tasks where accuracy is critical.

Interpretability and Explainability are also important properties for the tuning of models and for cleaning training data. With a larger dataset, it becomes more difficult to filter harmful examples from the dataset or to find a sample subset of interest. Explainable and interpretable models can be analyzed after training and help improve the training data for future iterations. Explainability is helpful for improving or “debugging” unhelpful responses even when the original training dataset is not available. This can help users prevent misunderstandings or inaccuracies in the model and use it more effectively and securely.

1.3. Updatability

Another limitation of modern LLMs is the difficulty of updating a trained model with new data or of removing specific data samples after the model has been trained. Since the parameters of a trained model are practically impossible to decipher perfectly, it is difficult to directly update them for a desired effect without possibly corrupting previously learned data points. Like all neural network based architectures, LLMs are usually trained and updated using gradient descent. It is possible to optimize a neural network further on additional data points and even give negative feedback, for example in RLHF (Reinforcement Learning with Human Feedback) [11], which is common practice when fine-tuning models for specific tasks, however there can be no certainty that the update will not overfit, corrupt previously learned knowledge and introduce hallucinations without retraining the model on the complete updated data set.

—

We recognize fundamental shortcomings of neural networks when it comes to interpretability, reliability and updatability, qualities that are critical for a secure and effective application of statistical models for language processing. As a first step to develop a language model able to satisfy these requirements more successfully, we set out to lay theoretical foundations for a rule-based, symbolic language modelling framework where reliability, interpretability and updatability come as natural.

2 Related Concepts

First, we are going to explore related concepts and research relevant to our topic. This chapter provides an overview on related language modelling techniques, interpretability of language models and grammatical inference.

2.1. Language Modelling

Language modelling is generally used to provide algorithms with an understanding of natural language and generate human-like texts.

Definition 2.1 Language Model

Let Σ denote an alphabet of tokens. A language model M assigns a string of tokens $w_1, \dots, w_n \in \Sigma^n$ an estimated probability $P(w_1, \dots, w_n)$ or $P(w_i \mid w_1, \dots, w_{i-1})$ for all $1 \leq i \leq n$, based on distributions seen in a training corpus $T \in \Sigma^N$ of length N .

The language model considers a string of tokens $w = w_1, \dots, w_n$ as a time-discrete stochastic process $X_1, \dots, X_n$, where each token position is a random variable over an alphabet or vocabulary Σ . The model estimates the probability of a given string w by predicting the assignments to these random variables. We write $P(X_1 = w_1, X_2 = w_2, \dots, X_n = w_n)$ or $P(w_1, \dots, w_n)$ for the probability of the tokens $w_1, \dots, w_n$. From the chain rule of probabilities we know that [12]:

$$P(w_1, \dots, w_n) = P(w_1) * P(w_2 \mid w_1) * \dots * P(w_n \mid w_1, \dots, w_{n-1})$$

This means that a language model both models the conditional probabilities $P(w_i \mid w_1, \dots, w_{i-1})$ for all $1 \leq i \leq n$ and the probabilities of given sequences. Another way to see this relationship is the definition of conditional probability:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Which can be applied to sequences of tokens in the following way:

$$P(w \mid w_1, \dots, w_{i-1}) = \frac{P(w_1, \dots, w_i)}{P(w_1, \dots, w_{i-1})}$$

A basic model to estimate these probability distributions is the *n-Gram model*.

Definition 2.2 N-Gram Model

An n -gram of order n is a string w^n of n tokens. An n -gram model stores counts of all n -grams appearing in a given corpus $T \in \Sigma^*$. A count function $C(\cdot) : \Sigma^n \rightarrow \mathbb{N}$ maps each n -gram to the number of times it occurs in T . The probability of a token w_i following $n - 1$ given tokens can then be calculated using the counts for n -grams of order n and $n - 1$:

$$P(w_i \mid w_{i-n+1}, \dots, w_{i-1}) = \frac{C(w_{i-n+1}, \dots, w_i)}{C(w_{i-n+1}, \dots, w_{i-1})}$$

Intuitively, an n -gram model counts the portion of occurrences of the $(n - 1)$ -gram in the corpus which have been followed by the token w_i . This way it estimates the token at position i from the given context of $n - 1$ tokens.

N -grams are however limited by their order n in how much preceding context it can include during prediction. Relationships beyond this context window are not captured by a single n -gram model, which is one of its major drawbacks, as natural language contains long-range dependencies.

Increasing the value of n can quickly increase the number of counts having to be stored, as the number of n -grams to be counted can grow exponentially. The number of possible n -grams over an alphabet Σ of size $|\Sigma| = k$ is given by k^n . Although a corpus T of length N can only contain at most $N - n + 1$ n -grams for a given length n , large corpora are long enough for the number of possible n -grams to be counted to become unpractical.

To precisely model the probability of a token given an arbitrary context, n -gram models would need to store n -grams of arbitrary length. As this would often require a lot of space to store the various possible n -grams, usually the n -gram length is limited. It is possible to estimate the true probability of each n -gram given in the corpus by only looking at a limited context size. This assumes the Markov property on the stochastic process and helps limit the memory required for the n -grams:

Definition 2.3 Markov Property

A stochastic process $X_1, \dots, X_N$ is said to possess the Markov property of order n if:

$$P(X_i \mid X_1, \dots, X_{i-1}) = P(X_i \mid X_{i-n}, \dots, X_{i-1})$$

That is the probability of the i -th state only depends on the past n states. A stochastic process with this property is also called an n -th order Markov Chain.

A Markov chain of order 1 is also called *memoryless*, as its next state only depends on the current state. N -gram models make the assumption that the stochastic process of the language they are modelling possesses the Markov property and that the probability of a token conditioned on the entire context can be approximated with a limited context window:

$$P(w_i \mid w_1, \dots, w_{i-1}) \approx P(w_i \mid w_{i-n+1}, \dots, w_{i-1})$$

2.2. Distributional Semantics

In the research field of distributional semantics, the similarity between linguistic items is defined as the similarity of their respective context distributions in a given text corpus. The distributional hypothesis proposes that “words which appear in similar contexts are similar in their meaning” or “words are characterized by the company they keep”.

Various methods for capturing context information for language modelling have been developed, most notably Word2Vec [13] has been used to learn *word embeddings* from the contexts the words occur in. Word embeddings being high-dimensional real-valued vectors, optimized using gradient-descent, which can easily be used for similarity comparison and arithmetic calculations in a hyper-dimensional semantic space. They represent words based on their distributional semantic meaning and allow for relatively robust predictions of surrounding contexts, however as iteratively optimized hyper-dimensional vectors they are usually difficult to interpret while also being lossy representations of the context.

2.3. Attention Mechanism

N-gram models have since been superseded by recurrent neural networks (RNNs), LSTM (Long-Short Term Memory) networks and then Transformers [14], a self-attention based neural architecture. Transformers are designed to *transform* a sequence of input tokens into another by incorporating context information from the input tokens into each output token. Transformers use word embeddings to represent individual tokens and during training, they learn *attention weights* between all directed pairs of tokens in the vocabulary. The attention weights allow the model to more efficiently incorporate context information into its latent representations by learning the relative importance of words in the context.

Definition 2.4 Scaled Dot-Product Self-Attention

Let $X \in \mathbb{R}^{n \times d}$ be the matrix of d -dimensional word embedding for the n input tokens. Further, let $W_K \in \mathbb{R}^{d \times d_k}$, $W_Q \in \mathbb{R}^{d \times d_k}$ and $W_V \in \mathbb{R}^{d \times d_v}$ be the *key*, *query* and *value* weight matrices, mapping each word embedding x_i to its respective key, query and value vectors k_i , q_i and v_i .

$$K = XW_K, \quad Q = XW_Q, \quad V = XW_V$$

Such that $K \in \mathbb{R}^{n \times d_k}$, $Q \in \mathbb{R}^{n \times d_k}$ and $V \in \mathbb{R}^{n \times d_v}$ contain k_i , q_i and v_i for all n input tokens in their rows. The self-attention mechanism computes the output embedding for each input token i as a weighted sum of all the value vectors j in V , where the weight is the dot product of the associated q_i and k_j vectors, scaled by the inverse root of the key and query vector dimension:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

The associated q_i and k_i vectors can be viewed as positions of the “sending” and “receiving”

endpoints in a “*link space*”, where the similarity of two vectors corresponds to the attention weight the first token assigns to the other. With this representation, tokens with similar contexts in the training data are assigned similar endpoints in the link space during training. This allows the transformer model to easily generalize from the training data and predict similar context even if they never occurred in the training data, simply based on the similarity of each token’s latent representation.

2.4. Grammar Induction

As opposed to neural networks, formal grammars are a rule-based approach to model a language. A formal grammar is generally defined as a set of *production rules* producing valid examples of a target language.

Definition 2.5 Formal Grammar

A formal grammar $G = (V, \Sigma, P, S)$ is represented by

- V , a finite vocabulary of symbols used in the production rules
- Σ , a set of terminal symbols, called the alphabet
 - $N = V \setminus \Sigma$, is the set of non-terminal symbols
- $P \subset (V^* \setminus \Sigma^*) \times V^*$, a finite set of production rules, mapping strings of symbols with at least one non-terminal to other strings of symbols
- $S \in N$, a starting symbol

Using a formal grammar, a language $L(G)$, which is defined as a set of strings, can be recognized and generated by a finite automaton. The task of learning such a grammar from a given set of examples of a language is called *grammar induction*. There are many approaches to this task from trial and error, over evolutionary algorithms to greedy inference of appropriate rules to represent the given examples.

Loss-less compression algorithms such as the Lempel-Ziv-Welch algorithm also qualify as grammatical induction, as they construct a grammar to represent the input with fewer symbols by deduplication. The Sequitur [15] algorithm has been used to reveal hierarchical structure in sequences such as text and music, by detecting repetitions in the data. Further, many modern neural architectures use grammar induction algorithms such as byte pair encoding [16] in their tokenization step to compress the training data while maintaining some awareness for the substructure of words. This shows that grammar induction is not only a means to reduce memory consumption, but also to identify significant structure of the input which can help modelling of the data.

Although much research in the field of grammar induction exists, we have not found a grammatical induction algorithm that is able to relate all frequent substrings of an input with their respective contexts in the input. As this is crucial information used by state-of-the-art language models such as the transformer, and since the context is a defining part of to the general concept of distributional semantics, we set out to build an algorithm to induce such a grammar.

3 Model Definition

This chapter describes the design of our context graph structure with which we seek to develop a novel language model that is completely reconstructable, human-readable and updatable.

The model is based on a hierarchical structure of n-grams with a minimal representation of a sub-string containment order, augmenting n-grams with their contextual relationships and reducing their storage size. Frequency counts of n-grams can be calculated to perform various stochastic predictions within the contextual neighborhood of different parts of text.

3.1. Context Representation

The key idea of the model is to represent all context relationships between frequent n-grams in a hierarchical deconstruction of the original corpus or input string. This approach is highly motivated by compression algorithms using a grammar or dictionary to represent their input while occupying less memory. These grammars do however not contain all context relationships between frequent n-grams and it is thus not possible to efficiently analyze them.

The goal of this chapter is to find the minimal set of edges necessary to relate all frequent n-grams in a context graph and assign each node in the graph a frequency count, which can be used to calculate probabilities for transitions or nodes in the graph.

We define the containment order on the set of all n-grams in the corpus of any length n . Represented as a graph, this relation will have all edges pointing from larger n-grams (super-strings) to the smaller n-grams (sub-strings) contained in them. Each n-gram or string is represented by a unique node connected to all of its sub-strings and their different parent contexts, giving a *directed acyclic graph* (directed edges, no cycles).

Definition 3.1 Sub-String Relation

The sub-string graph $G = (V, E)$ of a string $w \in \Sigma^*$ represent the **strict containment** order on sub-strings of w . Let $a < b$ denote the partial order on the set of sub-strings of w , “ a is a strict sub-string of b ”: We may write $a \leq b$ to allow for $a = b$. The nodes of the graph are given by the set of all sub-strings of w and the edges are given by the strict sub-string relation over these sub-strings:

$$V := \{v \in \Sigma^+ \mid v \leq w\}, \quad E := \{(a, b) \in V^2 \mid a < b\}$$

Note that V includes the original string w .

We will denote the string representation of a node $v \in V$ as $\text{string}(v)$ and refer to them as n-grams of degree $n = |\text{string}(v)|$

As a result, the graph of the sub-string relation contains all the edges directly connecting super-strings with all of their sub-strings.

For large corpora, this structure would grow far too large to be practical as the worst-case space complexity for storing the nodes is quadratic. The number of possible sub-strings of length n within a string of length N is $N - n + 1$. For all lengths n up to N , this amounts to $\sum_{n=1}^N N - n + 1 = \frac{N(N+1)}{2}$ many sub-strings to a string of length N for which nodes need to be stored.

For each node of length n the sub-string relation stores an edge to each of its substrings, of which there are $\frac{n(n+1)}{2}$ by the same argument as above.

The total number of edges will be the sum of all edges for every node of length n .

$$|E| = \sum_{n=1}^N (N - n + 1) \frac{n^2 + n}{2} = \frac{1}{2} \sum_{n=1}^N Nn^2 - n^3 + (N + 1)n$$

with $s(k) := \sum_{n=1}^N n^k$ and $s(3) = s(1)^2$ (Faulhaber's formula [17]) we can follow

$$\begin{aligned} 2|E| &= N^2s(2) - s(3) + N(N + 1)s(1) = N^2s(2) - \frac{(N(N + 1))^2}{4} + \frac{(N(N + 1))^2}{2} \\ &= N^2s(2) + s(3) = \frac{N^3(N + 1)(2N + 1)}{6} + \frac{N^2(N + 1)^2}{4} \end{aligned}$$

resulting in a worst case space complexity of $O(N^5)$ for the number of edges in the complete sub-string graph. We are going to show how to reduce the number of edges we need to store, while maintaining all the paths between substrings and keep sub-strings connected with their contexts.

3.2. Reducing the Number of Edges

We are only concerned about representing context relationships with this structure. We can perform a **transitive reduction** on the sub-string relation graph by only preserving the minimal set of edges needed to keep all paths between any two n-grams.

Definition 3.2 Transitive Reduction

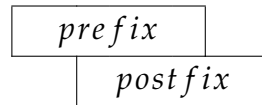
The transitive reduction $G_T = (V, E_T)$ of a graph G with no cycles is the result of removing all edges from G for which there exists a longer path connecting their endpoints. The graph G_T has the properties

- (i) There is a path between nodes u and v in G_T if and only if there is a path between u and v in G . The graphs have the same *reachability* relation.
- (ii) There is no graph with fewer edges than G_T fulfilling property (i)

In the transitive reduction of the sub-string relation of a corpus, edges can be omitted if there exists another path between the endpoints. This way we can still find all sub-strings of any given

node and can also reach all possible contexts of a string. The reachability of the transitively reduced relation is the same.

This allows us to store only a subset of the edges as we can reuse the connections from other related strings. The only downward edges that need to be stored for each node are to those sub-strings which can not be contained in any larger sub-string. Under the assumption that nodes for *all* possible sub-strings exist, there are only 2 sub-strings of each string with this property, the prefix and the postfix of length $n - 1$.



In the transitive reduction, still every sub-string is reachable from every node, by travelling over the paths iteratively. However, as we only have to store a constant number edges for each node, the number of edges in the sub-string relation is reduced to $N(N + 1)$ or $O(N^2)$.

3.3. Reducing the Number of Nodes

The goal of the model is to connect frequent sub-strings of the corpus with their various contexts and to derive probabilities for different contexts of a given string. For this we do not need to store dedicated nodes for n-grams which occur only in a single context, as it can be represented in the context node and storing an additional node would be redundant. Root super-strings represent training examples and will have no parents. A constraint to formalize this could be:

Nodes should not have exactly one parent.

This can however not always be fulfilled, as some frequent child nodes of a string may overlap, while other children do not have overlaps. There are multiple ways to deal with this, and we have chosen to create new nodes for the overlapping portions to avoid repeating surrounding, non-overlapping contexts. Now the constraint becomes:

Nodes must not have exactly one parent while having no overlapping children.

To define this more formally, we are going to make another extension to the structure with *string partitions*. Instead of only linking each node with the minimal set of smaller nodes required by the transitive reduction, we require each node to contain the minimal set of partitions of its string, represented using all child nodes from the transitively reduced containment graph.

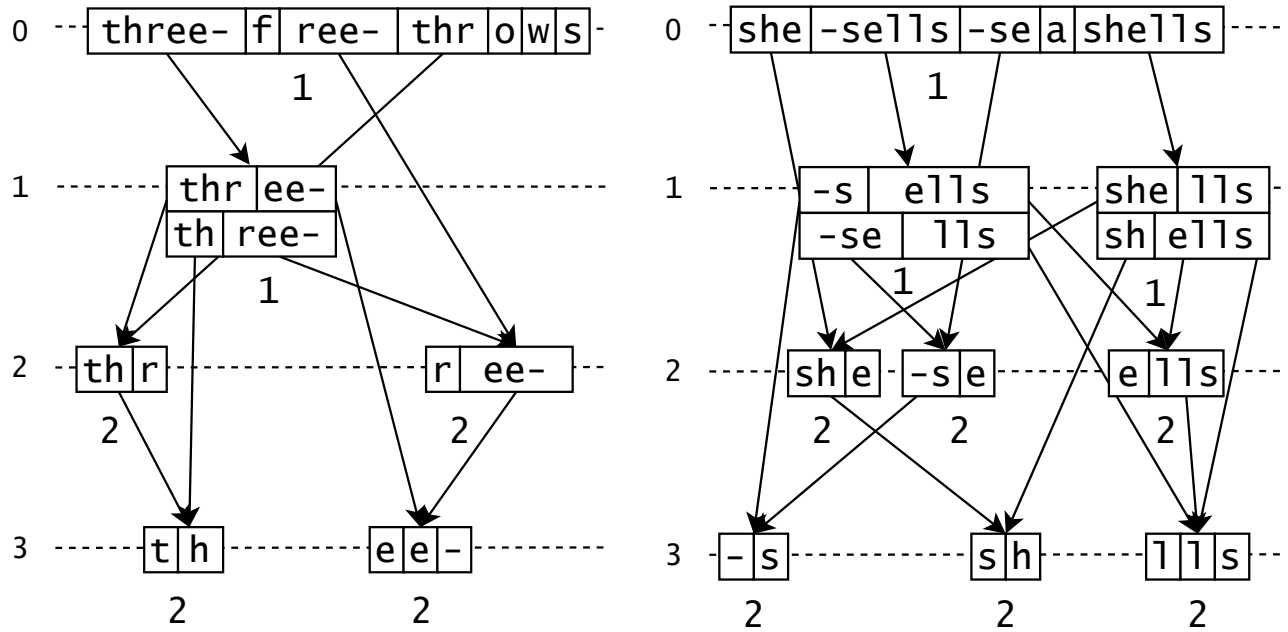


Figure 3.1: Substring partition graphs of the strings three-free-throws and she-sells-seashells. Nodes show partitions and frequency counts. Unigram nodes are omitted and levels are annotated by depth.

Definition 3.3 Partition Graph

Each node n contains a minimal set of partitions $\text{parts}(n)$, where each partition p_i is a list of n_i child nodes, together representing the entire string of n : $\text{string}(n)$:

$$p_i := (v_{i1}, \dots, v_{in_i}) \quad \text{such that} \quad \prod_{1 \leq j \leq n_i} \text{string}(v_{ij}) = \text{string}(n)$$

(where $\cdot$ denotes string concatenation)

in such a way that every direct child in the transitively reduced sub-string graph G_T is present in one of the partitions:

$$\forall c \in \text{children}_{G_T}(n) : \exists p \in \text{parts}(n) : c \in p$$

This means every node in the graph of partitions G_P can reach all of its sub- and super-strings, since the child partitions contain all the edges necessary for the transitive reduction of the complete sub-string relation.

Since one partition can not contain two overlapping children, we can follow that any node with overlapping children in G_T also has more than one partition. The nodes contained in the child partitions of each node contain all nodes necessary to reach every existing sub-string node and additionally some padding at the outer ends, so that each partition represents the whole string.

While this is not necessarily required, as the padding nodes are also reachable from one of the other nodes, it makes the formalism a bit simpler, as each partition represents the same string.

In this formulation the criterion for each node can be written as:

$$\text{parents}(\text{node}) \neq 1 \vee \text{parts}(\text{node}) \neq 1$$

When applying this, many nodes of the original substring relation are removed and our assumption from 3.2 that all sub-string nodes exist does no longer hold. In the partition graph, we can see that no border between two adjacent sub-string nodes can occur in two different partitions. Any sequence of nodes in any two partitions with a border at the same token location will always be expressed as a single larger node containing the sequences as its child partitions instead, following the constraints from the transitive reduction 3.2. By removing those nodes that do not follow the above constraints, it becomes impossible for any two partitions to have a node border at the same token position.

From this follows that any node length n can have at most $n - 1$ borders between child nodes in all of its partitions and thus a number of child nodes in the order of $O(n)$. This gives us an upper bound for the number of edges in the partition graph:

$$\sum_{n=1}^N (N - n + 1)n = \sum_{n=1}^N Nn - n^2 + n = N^2s(1) - s(2) + s(1) = O(N^4)$$

This upper bound however ignores the fact that a higher number of nodes generally allows for more compression and ultimately results in less edges per node, illustrated by the extreme case where each node can have at most 2 significant children in the transitive reduction graph 3.2. Such an edge space complexity can only be reached if there is a minimal amount of repetition in the input, which would also reduce the number of total nodes.

3.4. Maximum number of nodes over an alphabet

The total number of possible strings length n which can be built over an alphabet of k symbols is k^n . This means that for small n , the number of possible individual nodes may much smaller than is indicated by the number of possible sub-strings: $N - n + 1$, as they are limited by the number of unigrams or atomic tokens k . With respect to this, the equation for the total number of possible nodes in a string length N over k symbols becomes:

$$|V| = \sum_{n=1}^N \min\{k^n, N - n + 1\}$$

Both the functions are monotonic with respect to n , k^n being an increasing function and $N - n + 1$ is a decreasing function.

To sum only the minimum value of both functions, we can split the sum into two parts at their intersection point n^* , where

$$k^{n^*} = N - n^* + 1 \Leftrightarrow \log_k(k^{n^*} - n^*) = \log_k(N + 1) \quad (3.1)$$

With the intersection point, can then express the sum as two parts, summing each side of the intersection independently:

$$|V| = \sum_{n \in [1; n^*]} k^n + \sum_{n \in (n^*; N]} N - n + 1 \quad (3.2)$$

Because equation 3.1 is a transcendental function, there is no algebraic solution for n^* , but we can get a sufficient estimate by utilising that

$$\log(s + t) = \log(s) + \log\left(1 + \frac{t}{s}\right) \approx \log(\max\{s, t\}), \text{ if } |s - t| \gg 0$$

which allows us to rewrite equation 3.1 as

$$\log_k(k^{n^*} - n^*) = \log_k(N + 1) \approx \log_k(\max\{k^{n^*}, -n^*\}) = n^*, \text{ if } k^{n^*} \gg -n^*$$

giving an approximation of $n^* \approx \log_k(N + 1)$ for large N , sufficient for our purposes. We have verified this intersection point numerically using Newton's method for approximating roots of non-linear equations.

Then, we approximate equation 3.2 with its integral form

$$\begin{aligned} |V| &\approx \int_1^{n^*} k^n \, dn + \int_{n^*}^N (N - n + 1) \, dn \\ &= \left[\frac{k^n}{\ln(k)} \right]_1^{n^*} + \left[(N + 1)n - \frac{n^2}{2} \right]_{n^*}^N = \frac{k^{n^*} - k}{\ln(k)} + \frac{N^2}{2} + N - (N + 1)n^* + \frac{n^{*2}}{2} \end{aligned}$$

and replacing $n^* = \log_k(N + 1)$

$$|V| \approx \frac{N + 1 - k}{\ln(k)} + \frac{N^2}{2} + N - (N + 1)\log_k(N + 1) + \frac{\log_k(N + 1)^2}{2}$$

giving an asymptotic order of $O(N^2 + \log_k(N)^2)$ for the total number of possible sub-strings for a corpus of length N over an alphabet of k tokens. Giving an even lower upper bound for the worst case for the number of nodes required to represent a specific string of length N over k different tokens.

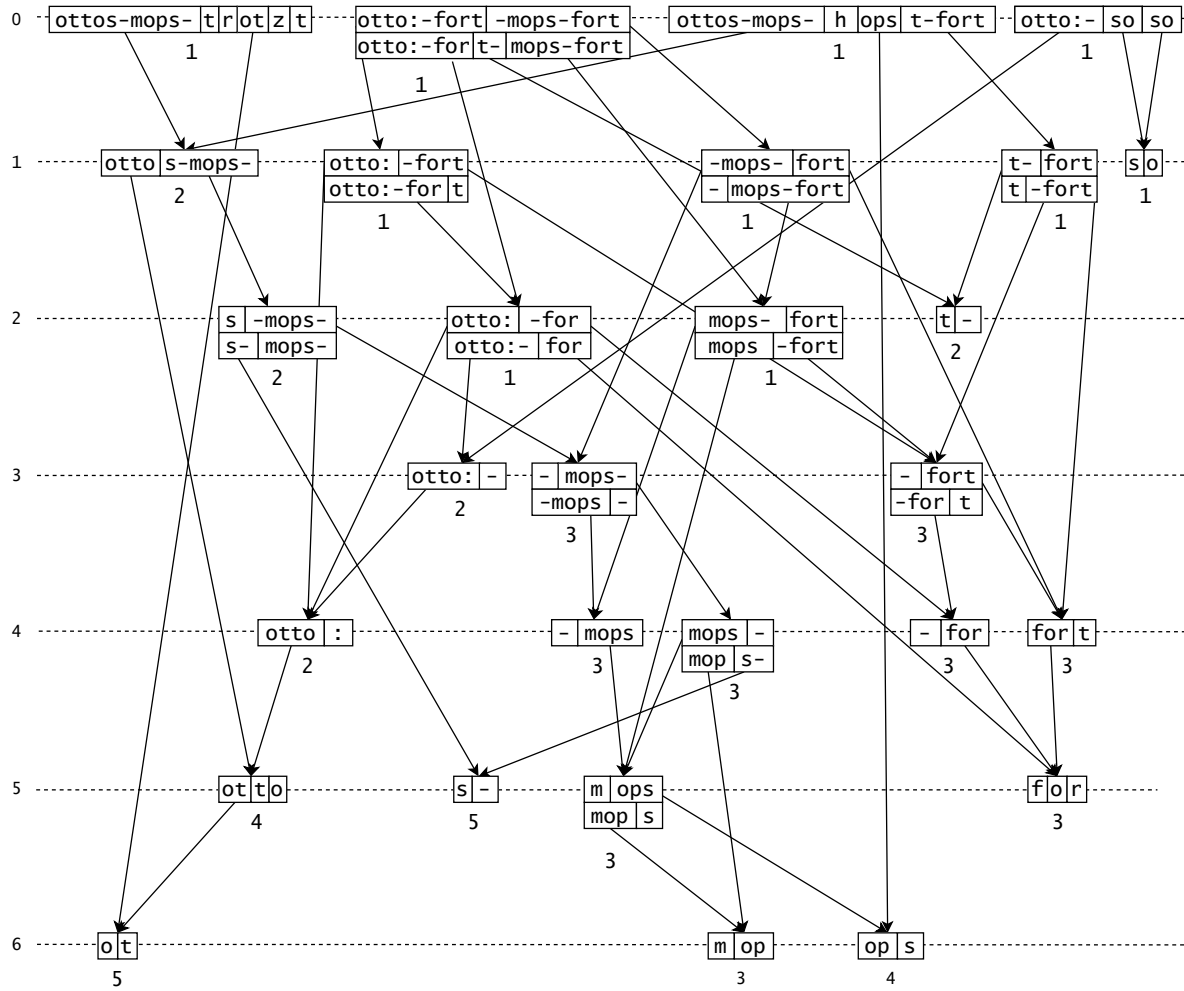


Figure 3.2: A complete transitively reduced sub-string partition graph of four example sentences. Nodes show partitions and frequency counts. Unigram nodes are omitted and levels are annotated by depth.

3.5. Reconstructing Training Data

Of course, it is generally possible that the same sub-string appears at different positions in a larger string, for example *prefix* = *postfix*. To account for this in the representation, we associate each edge in the relation with the relative position of the child in the super-string. The resulting graph has edge labels in the form of sets of these positions.

Definition 3.4 Offset Relation

The offset relation $\text{offsets}(e) : E_T \rightarrow \mathcal{P}(\mathbb{N})$ assigns each edge of the reduced sub-string relation G_T the set of offsets where the sub-string occurs in the super-string.

$$\text{offsets}(x, y) := \{|a| \mid \forall a, b \in \Sigma^* : \text{string}(x) = a \cdot \text{string}(y) \cdot b\}$$

where $\text{string}(\cdot)$ gives the string associated with a node in G_T . Each offset is defined as the number of tokens preceding the sub-string in the super-string.

With the offset relation in place we are able to reconstruct the original string from the reduced sub-string relation G_T , as we now know where each sub-string needs to be placed. Because nodes in G_T must be connected to all of their sub-strings, we are able to reach each of the tokens represented by the node. With the offset relation, providing us with a relative position of each sub-string, we can also derive the absolute position of each token and reconstruct the input.

For any node v we can reconstruct its corresponding string $\text{string}(v)$ if we know the strings of its children. If a node has no children it shall represent a single token, drawn from a dictionary. These nodes represent unigrams, where $|\text{string}(v)| = 1$. Any larger nodes must have these nodes as sub-strings and thus a path to these unigram nodes. Each path is annotated with offsets on each edge, and thus we can reconstruct the token for each position of a node's string.

3.6. Counting n-gram Frequencies

For statistical predictions about surrounding contexts frequency counts can be used to calculate the probability of each frequent context:

Definition 3.5 Frequency Counts

$\text{count}(v) : V \rightarrow \mathbb{N}$ assigns each string in the sub-string relation G_T the number of its occurrences in the corpus.

The frequency counts can be used in an N-Gram model to make predictions about surrounding tokens of a given string:

$$P(c \mid n) = \frac{\text{count}(c)}{\text{count}(n)}$$

The counts represent the number of occurrences of each sub-string in the training corpus T . Each of these occurrences in T can be represented by a position index able to reference any position in the given corpus. For example, a corpus consisting of a single string, could reference each sub-string's occurrence by its offset p from the start of the string. A corpus composed of multiple separate strings could use a two-dimensional index (s, p) of string index and position to reference the occurrence of some sub-string.

In a given graph these occurrences can easily be counted by passing all occurrences from top to bottom. It requires a bit of care since multiple different contexts can point to the same child at the same position (for example `lls` is a child of `-sells` and `ells` but in both instances occurs at the same position), however this is easily handled by tracing the global position for each node and only counting unique instances.

Counting Algorithm A simple algorithm to calculate the counts for each node can be given by a recursive breadth-first traversal of the graph, counting every unique occurrence for each node and passing them on to the child nodes.

For each node n we collect a set of occurrence positions O_n , where the root nodes are assigned a single position: $O_s := \{(s, 0)\}$. Every child node will now construct its set of occurrences from each of its parents occurrences and its offset in the parent:

$$O_n := \{(s, i + r) \mid (s, i) \in O_p, p \in \text{parents}(n), r \in \text{offsets}(p, n)\}$$

In words, the occurrences of a node n are given by its relative positions in the occurrences of all of its parents. The count of each node is then given by the number of occurrences: $\text{count}(n) := |O_n|$.

—

Now that we understand the structure of the model, we can define the algorithm for constructing it from a raw text corpus.

4 Induction Algorithm

In this chapter we describe the fundamental algorithms required to implement the automatic induction procedure (described in 4.3) and how to construct the context graph defined in the previous chapter.

The induction procedure is divided into three main operations:

- **Search** Parses an input string to a parse tree in the context graph corresponding to the largest matching prefix. Produces a sub-graph annotated with positions to locate the exact range of the match.
- **Insert** Uses the parsed sub-graph from the search step to split and modify the context graph to guarantee a dedicated node for the matched string.
- **Read** Repeatedly inserts nodes for largest known sub-strings found in an input string and consequently constructs a node representing the entire input string.

4.1. Search

The search algorithm finds the location of a given input string (called *query*) in a given context graph structure. The entire query may not exist in the graph, therefore only the largest matching prefix is located. The algorithm constructs a *trace graph* containing the nodes and edges that were traversed during search in the complete graph. The trace graph is an augmented sub-graph of the original graph which is rooted at the smallest vertex containing all the matching string with paths to the locations of the beginning and the end of the matched string in the vertex.

Definition 4.1 Trace Graph

The trace graph T_G of string $w \in \Sigma^*$ in context graph G is a rooted sub-graph found in G , representing w . The edges of T_G contain at least one path starting from the first node, passing through the root node and ending in the last node representing sub-strings of the query (mirroring traversal order of the search algorithm).

The edges can be divided into *upward* and *downward* edges, where upward edges point to larger nodes and downwards edges point to smaller nodes.

$$d(a, b) = \begin{cases} \text{up,} & \text{if } |a| < |b| \\ \text{down,} & \text{if } |a| > |b| \\ \text{undefined,} & \text{if } |a| = |b| \end{cases}$$

Nodes of the same size can not be in a strict containment relation and thus no edges between exist in the context graph.

Edges are annotated with a distance $\text{dist}(e)$ to the first token, relative to the nodes they connect.

$$\text{dist}((a, b)) = \begin{cases} \text{prefix length before } w_0 \text{ in } a, & \text{if } (a, b) \text{ is upwards edge,} \\ \text{distance from } w_0 \text{ to beginning of } b, & \text{if } (a, b) \text{ is downwards edge} \end{cases}$$

The distance values are later used to determine the relative positions a node needs to be split at to construct new nodes from partial overlaps with other nodes.

To describe the search algorithm, let us assume there is an existing context graph structure that is correct according to our definition 3. From our definition, we can derive assumptions about the structure at every operation of the algorithm. For a base example of the graph structure we can give the graph containing a single sequence without repetitions.

The search query is given as a string of vertex indices $q \in I_V^*$, representing a string of either unigram tokens or larger n-gram nodes, meaning we are not limited to raw string search queries. The query is the pattern we are trying to find in the graph.

The search algorithm maintains a queue of states with which it traverses all parse states in a breadth-first manner. The states contain information about the locations in both the *query* and the graph structure G to be compared. While traversing the graph, the algorithm populates the trace graph T with new edges to each matched sub-string. The edges are annotated with relative positions which are later used in the insertion operation 4.2 to split any partially matching vertices for further processing.

The algorithm can be understood as a parsing algorithm, matching the “rules” of graph structure on the query. This is a more intuitive description:

1. Start at the first vertex index in the query q_0 and begin upwards traversal in G in a breadth-first manner adding next nodes to a queue
2. Stop at each parent where there exists a successor to the previous child edge, i.e. the parent describes a context of q_0 with consecutive tokens
3. Compare the successive vertex n with the following token in the query q_n . There are three cases for n and q_n :
 - equal: a matching transition has been found
 - same length but different nodes: a mismatching transition has been found
 - different lengths: continue search by comparing the prefixes of the larger vertex l with the smaller vertex s . If s is a prefix of l , there must be a path from l to s .

If both nodes have different lengths (l and s exist), the matching algorithm continues to match the prefixes of l with s . Once we encounter a prefix p of l that is smaller than s , we can assume, based on the correct structure of l , that s does not exist completely as a prefix of l (because there is no path from l to s at the beginning). However, it is still possible that the prefix p and s both share a smaller prefix before their strings differ. To find these prefixes, s becomes the new l and p becomes the new s and the algorithm repeats until s and l are the same length, which is

bound to happen as all vertices have a smallest prefix of length 1, when a decision can be made. The result of this matching is therefore always a path from the parent node n to a matching or mismatching prefix for both the query and the search cache.

This algorithm is repeated on larger and larger root nodes, until a root arrives at a mismatch with the query. During iteration, edges are added to the trace graph if they are guaranteed to be part of the matching sub-graph structure. Such is the case when a parent node has been found to match until its end and when a mismatch has been found. The collected paths and relative positions can then be translated to token locations in parallel partitions in each node, constructing all possible paths from the root node to the respective beginning and end tokens.

The resulting trace graph contains all the possible paths from the start vertex (q_0) to the root, aswell as from the root to the last vertex matching with the query. Each edge in the trace graph is an edge from the containment relation, annotated with a distance from the beginning of the query. These annotations are added during parsing and allow us to calculate the relative location of each border (start/end) inside each vertex.

In the next step, this information is used to split each vertex into parts which are then recombined into new vertices.

4.2. Insert

To construct the grammar for a raw given string, vertices for new sub-strings must be inserted into the grammar. New nodes must uphold the invariants of a correct grammar for this to produce a new consistent grammar.

From the definitions in chapter 3, we derive these invariants all nodes must uphold:

- **Digram Uniqueness**
No contiguous sequence of rule symbols may appear more than once across all nodes.
- **Boundary Uniqueness**
No boundary between rule symbols may appear at each token position more than once across all partition rules of a node.
- **Containment Path Completeness**
There exists a path from every node to all of its sub-string nodes.

The general principle of creating new nodes under these constraints is to reuse as many nodes from the existing grammar as possible, and *splitting* any partially matching nodes up into an “inside” and an “outside” partition.

The resulting partitions for each node are then *joined* together from smallest to largest to represent a single node for the respective string.

4.2.1 Splitting Nodes

A trace graph obtained from the search algorithm locates all the positions where nodes are intersected by the matched string. These are the nodes containing parts of the matched string which need to be reused.

The trace graph consists of two sets of “upwards” and “downwards” paths over grammar nodes intersected by the matching string. The paths both connect with the *root node* representing the smallest string containing the entire matched string. Each set of paths represents the begin and end boundaries of the match respectively.

Each partially matching node is visited by at least one path of a boundary, at a specific token position. Thus, the trace graph annotates every intersected node with positions where the given node needs to be split into partitions.

The *root partition* is formed by the boundaries in the root node and represents exactly the matching string as a partition in the root node. Therefore, the root node can at most be split at two positions (once for the begin boundary and once for the end boundary).

Smaller child nodes can only contain partial matches and may generally be split at every possible position. An example of a node requiring a split at every of its n positions would be an n -gram of a single repeating token. This kind of node would exist at every single shift in a larger node repeating the token a greater number of times:

aaaaaa ::= aaaaa,a	aaaaa ::= aaaa,a	aaaa ::= aaa,a
aaaa,aa	aaa,aa	aa,aa
aaa,aaa	aa,aaa	a,aaa
aa,aaaa	a,aaaa	
a,aaaaa		

Figure 4.1: Example grammar illustrating the same node occurring at multiple different offsets.

In this example, a split of aaaaaa (a^6) after 1 and 3 tokens would incur splits in aaaa (a^4) at its relative positions 1,2 and 3 because it exists in aaaaa (a^5) with a shift of 1, so when a^5 is split at position 3, it requires a split at position 2 in a^4 .

The algorithm to split the nodes consistently with the grammar’s invariants works by iteratively joining the partitions of smaller nodes into the surrounding context in their parent to create the larger partitions of the parent.

An outer loop traverses all the nodes from the bottom up, starting at the leaves and following their edges upwards in the trace graph. Each edge represents a split of a child node with an “inside” and “outside” half. When these splits have already been computed for all children of a node, we can locally split the node at all requested positions.

$xyabcdefxy := xy,abcdef,xy$	$cde := c,de$	$ab := a,b$
$abcdef := ab,cde,f$	$def := de,f$	$de := d,e$
$ a,bc,def$	$bc := b,c$	$xy := x,y$

Figure 4.2: Excerpt from example grammar

Given this grammar and the objective $SPLIT(abcdef, 4)$ we can see that we require the child splits $(cde, 2)$ and $(def, 1)$. Assuming these have already been computed as

$$SPLIT(cde, 2) = (cd, e) \quad \text{and} \quad SPLIT(def, 1) = (d, ef)$$

We can now collect the split halves of all of $abcdef$'s rules, by placing each child split next to its one-sided context from its rule:

$$\begin{bmatrix} ab, cde, f \\ a, bc, def \end{bmatrix} \longrightarrow \begin{bmatrix} ab, cd \\ a, bc, d \end{bmatrix} \quad \begin{bmatrix} e, f \\ ef \end{bmatrix}$$

These new rules can then be joined into new nodes, yielding a representation for each half of the split which can be used in the next steps of the algorithm. Keep in mind that in this example we only split the node at one position, while in practice, it is possible that each node requires multiple splits.

In any case, splitting rules and joining them into new nodes may create structures which are not consistent with the required invariants. In the example above we would violate the digram uniqueness and rule utility invariants if we inserted the new rules:

- The left collected half contains the pattern a, bc, d , repeating the sequence a, bc from the second rule of $abcdef$, which violates *digram uniqueness*.
- The right half contains a rule of a single node, which renders the other rules obsolete, as they must already be present in that given single node, which violates *rule utility*.

The other invariants, path completeness and boundary uniqueness, are satisfied, as we simply reused the nodes and boundaries in previously consistent rules at the same locations and no new larger nodes have been created.

First, let us address how to handle rule utility and digram uniqueness. Rule utility is straightforward, as it is easy to detect if a new set of rules contains a rule with only a single node that can be used instead. Digram uniqueness requires us to put in a little more effort.

4.2.2 Removing Repetitions

When we split rules to reuse their parts, we may always end up with partitions repeating sequences already present in the original rules.

$$\text{SPLIT} \left(\begin{bmatrix} \text{a, bc, def, gh, ij, klm, no} \\ \text{ab, cde, fg, hi, jk, lmn, o} \end{bmatrix}, \{4, 12\} \right) \longrightarrow \begin{bmatrix} \text{a, bc, d} \\ \text{ab, cd} \end{bmatrix} \quad \begin{bmatrix} \text{ef, gh, ij, kl} \\ \text{e, fg, hi, jk, l} \end{bmatrix} \quad \begin{bmatrix} \text{m, no} \\ \text{mn, o} \end{bmatrix}$$

This generally occurs when a partition created by one or more splits spans more than two node boundaries in a rule, because then the partition contain at least a whole digram from the original rule.

A split position in a node is defined by:

- a **token offset position**, describing the number of tokens preceding the split in the respective node
- for **every rule** in the node:
 - an **index position**, describing the symbol in the rule that the split occurs in
 - an **inner offset**, describing the token position inside the child node pointed to by the index position

Split positions may look like this:

$$\text{SPLIT_POS} \left(\begin{bmatrix} \text{ab, cde, g} \\ \text{a, bc, def} \end{bmatrix}, 4 \right) = \left(4, \begin{bmatrix} (1, 2) \\ (2, 1) \end{bmatrix} \right) \quad \text{or} \quad \text{SPLIT_POS} ([\text{d, ef}], 1) = (1, [(1, 0)])$$

As shown in the example, split positions may of course also have an inner offset of 0 in one of the rules. In that case they exist at a boundary already present in that rule, and the split is called a *perfect split* in the respective rule in the node. Due to the boundary uniqueness invariant a split can only be perfect in a single rule of a node. When perfect splits participate in forming a partition, the boundary they occur on is also spanned by that partition and may contribute to a repeated digram being formed. For example in

$$\text{SPLIT}([\text{ab, cd, ef, ghi}], \{2, 7\}) = \{[\text{ab}], [\text{cd, ef, g}], [\text{hi}]\}$$

The sequence $[\text{cd, ef}]$ is repeated in the result of the splits and has to be handled to avoid a repetition before inserting it into a node.

To handle these repeated sequences within requested partitions, they need to be replaced by a single node in the original rule and the new rule. This means they require an extra partition, which is called an *inner partition*, as it occurs within an outer partition. The splits of an inner partition always occur on existing boundaries in the same rule and are thus always perfect in the same rule.

To create nodes for these inner partitions consistently, we need to treat them the same way as regular partitions which are created from split positions derived from the trace graph and join splits from all rules of a node.

The split positions for the inner partitions need to be included before joining any partitions, otherwise not all nodes below a node with an inner partition would know about the splits they need to perform. Thus, we need a *complete* step before actually performing the splitting and joining of partitions, to add any missing split positions to the trace graph.

The *complete* algorithm simply walks top-down through every node in the trace graph and adds the splits for any inner partitions within the partitions already being requested. Inner partitions from top nodes may of course induce additional inner partitions in bottom nodes, which is why a top-down iteration is required.

To differentiate between splits created for inner partitions and partitions requested by larger nodes, we will call them “inner splits” and “top splits” respectively. As a top split may also be perfect, it may also participate in an inner partition, and can also be considered as an inner split. However, it is only the top splits forming the partitions to be passed upwards to the larger nodes and the inner partition are merely used to form these “top partitions”.

4.2.3 Joining Partitions

After the inner splits have been added to the trace graph, no partitions can contain any unknown inner partitions and the partitions are ready to be joined. The joining procedure has to join all partitions in the trace graph from smallest to largest to ensure all newly created nodes to be included in larger nodes. This bottom-up approach applies first on the graph level and then on the individual node level. An outer loop visits all nodes from the bottom up, joining their partitions and thus making them available for their top nodes. Within each node, the partitions are also created starting with the smallest and joining them into the largest partitions requested by their top nodes. When a node is being processed inside the outer loop, it can assume all top splits for their children to be available.

When each node is processed, the algorithm is able to request all the partitions from child nodes and can construct partitions formed by any pair of split positions (including the node’s borders). It starts by joining all the inner partitions and replacing the respective sequences in the rules with them. We are going to look at an example for the objective

$$\text{SPLIT} \left(\left[\begin{array}{l} a, bcd, ef, gh, ijk, lm, no \\ ab, cde, fg, hi, jkl, mn, o \end{array} \right], \{3, 10\} \right)$$

After augmenting the inner splits, the ordered representation of the set of splits becomes

$$(3, 4, 5, 8, 9, 10, 11, 12)$$

Using the available bottom splits we can construct a partition of the rules between each of these positions:

$$\left[\begin{array}{l} a, bc \\ ab, c \end{array} \right] \quad [d] \quad [e] \quad [f, gh] \quad [i] \quad [j] \quad [k] \quad [l] \quad \left[\begin{array}{l} m, no \\ mn, o \end{array} \right]$$

These representations can be inserted as new nodes into the grammar and these nodes will be stored in a *range map* for the runtime of the joining procedure for the current node. The range map makes individual partitions accessible by their range in the ordered split position space (with 0 and the index after last referencing the node borders):

$$\text{range_map}[(0, 1)] = \text{abc} \quad \text{or} \quad \text{range_map}[(3, 4)] = \text{fgh}$$

It is possible to encounter partitions formed from two perfect splits, which allows us to modify the original rules of the node directly and replace the given range in the rules with the joined partition. So far this applies for the partition for *mno*, which forms a perfect partition with the split 8 and the right node border. We can replace the sequence in the original rule with the joined partition.

After all partitions of cell width 1 (referring to the number of cells created by splits being spanned) have been joined, we continue joining partitions of cell length 2. In this case we can not only consider the rules in the node, but also need to include all the smaller partitions within the same range into the representation for the new node. For the range (2, 4), we retrieve the partitions for ranges (2, 3) and (3, 4) from *range_map* and add them to the split partition (2, 4) retrieved from the rules:

$$[e, fgh] \cup \begin{bmatrix} ef, gh \\ e, fg, h \end{bmatrix} = \begin{bmatrix} e, fgh \\ ef, gh \\ e, fg, h \end{bmatrix}$$

Because the split at index 3 is perfect in the second rule, the representation contains a duplicated boundary. This needs to be expected when using partitions which were created from one perfect and one non-perfect split, in this case the partition (3, 4) or *fgh*. However, as the joined partitions we retrieved from *range_map* must always contain nodes at least as large as in the original rules, as we created them by joining nodes from the rules, we can safely discard any rules extracted from the node and give the rule retrieved from *range_map* precedence, producing

$$\begin{bmatrix} e, fgh \\ ef, gh \end{bmatrix}$$

Again we encounter perfect partitions, which we replace in the original rules. The perfect partitions of length 2 are *efgh*, *fghi* and *lmno* and *mno*. After inserting them, the rules of the node *abcdefghijklmno* become

$$\begin{bmatrix} a, bcd, efgh, ijk, lmno \\ ab, cde, fghi, jkl, mno \end{bmatrix}$$

We can proceed joining the larger partitions this way, until we encounter partitions which can not be contained in any of the top partitions anymore. We only need to join partitions which are required by top nodes and not those which may be formed by inner splits but exist outside those top partitions. An example of such a partition is (0, 8), which is not contained in any of the partitions formed by the top splits with index 1 (with token position 3) and 6 (token position 10), namely the partitions (0, 1), (1, 9), (0, 6) and (6, 9). Generally a partition (*x*, *y*) only needs to be created if there is at least one top split *s* where

$$x \geq s \wedge y \geq s \vee x \leq s \wedge y \leq s$$

meaning, there is at least one top split on either side of the whole partition. As the node borders do not count as top splits, the partition $(0, 8)$ does not satisfy this condition and must not be joined.

To finalize the joining procedure on a single node, the joined partitions for every top split have to be inserted into the rules of the node. In case any of the top splits is perfect, the respective rule with the matching boundary is to be replaced, as the top partitions span every other boundary in the rule and have been constructed to contain any other smaller partitions of the node.

Finally, any rules only containing nodes completely contained in nodes from other partitions can be dropped without violating any constraints, as they will still be reachable from those other nodes. For the example this leaves us with the rules:

$$\begin{bmatrix} abc, defghijklmno \\ abcdefghij, klmno \end{bmatrix}$$

4.3. Read

Now that we know how to modify the grammar to represent a part of a known string with a dedicated node, we still need to understand how to add representations for entirely new strings into the grammar. A new string can contain unknown tokens and unknown sequences of known tokens. Let Σ be the alphabet of input tokens and I_V be the index set of nodes in the grammar. Then an input sequence is an element $x \in (\Sigma \cup I_V)^*$ with $x = x_1, \dots, x_n$. A token or sequence is new or unknown when there exists no node in I_V representing the token or sequence.

A sequence like this can be parsed by a given graph G using the search algorithm which returns a trace graph for the largest matching prefix. This trace graph can then be joined using the insert algorithm to ensure there is a designated vertex for the matching prefix of the string. By repeating this operation and continuously updating the grammar to include larger and larger parts of the given input string, we can eventually represent any entire input string as a consistent node in the grammar.

Unknown tokens are easily processed as they can not be included in any larger nodes and can not yield any results in the parsing step. Such tokens are simply inserted into the grammar directly by which they become known in the consecutive sequence. The primary challenges arising for this objective are the handling of overlapping known nodes and the continuous update of the grammar while parsing the input sequence.

4.3.1 Recording overlapping Nodes

When parsing the input string, we may find nodes partially reaching into a later known sequence. This creates an overlap between two nodes which both need to be represented in the final partitions of the input string. An overlap occurs when the second sub-string starts before the first sub-string has ended.

These matches also need to be recorded to be able to fulfill the path completeness invariant for the final node. In principle *every* largest known sub-string has to be found, starting at any position. Conveniently, we can use the existing grammar and its invariants to reduce the search space significantly.

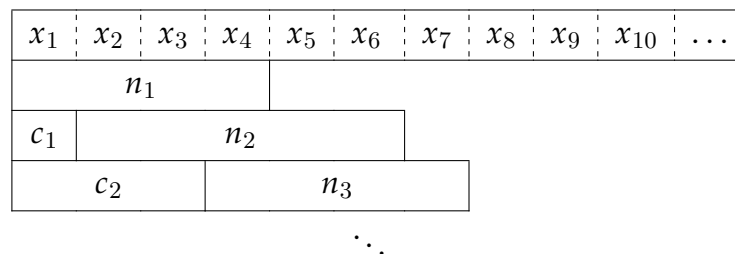
From the path containment invariant we can derive that, in general, every node in the grammar must contain the node for its largest postfix for which a node exists in the grammar. Any postfix that does not have its own node can not occur in any other parent or else these rules would violate the digram uniqueness requirement, as the same sequence would be given in multiple rules.

Therefore, we can find no overlap for any post- or prefixes which do not have a dedicated node. Thus we can find all candidate overlaps by simply traversing all single-node postfixes of the first match we have found.

We can modify the search algorithm to start with a postfix node instead of the first token of a search query, as it also works with node indices in its search query. The search query would simply be given by the candidate postfix followed by the input remaining after the initial match. Any parents found this way are essentially *expansions* of the candidate postfix into the remaining input string.

If the largest postfix does not have a parent matching with the remaining input sequence, we loosen the constraints and choose smaller and smaller postfixes of the current match, starting at later positions in the input string, until we find a parent expanding into the remaining input or reach the postfix with the length of one token without finding an expansion. In this case we can rule out that there is any overlapping nodes present for this node in the grammar and therefore also no overlapping sub-strings for this match.

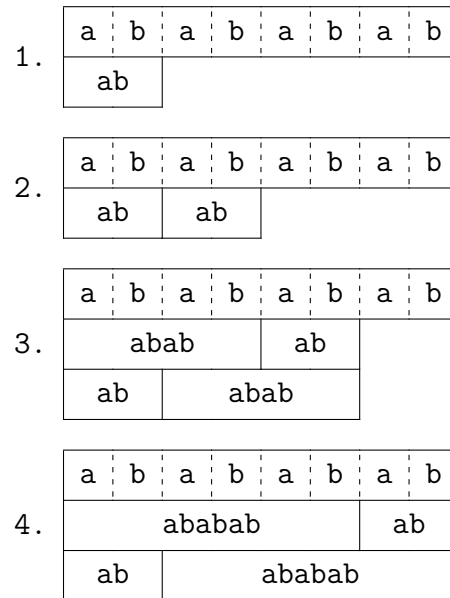
When overlaps are found, the process will record a chain of overlapping nodes represented in the input string. All the nodes in the chain share a child node as a postfix or prefix respectively, also referred to as the overlap between these nodes. Each overlap must skip at least one token and thus a chain of overlaps can at most be as long as the token width of the first overlapped node n_1 . As



all the nodes in a chain need to be included in the structure of the final result, each element in the chain seeds a new rule to be constructed. To use the rule in a new node, each rule has to constitute a partition of the string to be represented and the gaps appearing before the overlapping nodes need to be filled with the appropriate nodes, which we denote as the *context* of the respective overlap.

This is a notable point as this creates the first *new* digram in the induction process. This new digram has to be available immediately to the subsequent search calls inside the grammar. An example for why this is needed is given by the string abababab:

In the first steps we need to add the unknown tokens a and b , and recognize them as a digram ab . There can not be any known overlaps with this digram, as the tokens were unknown. As we read the first known token, the second a in step 2, we try to match its parents with the remaining input and find that ab expands into the remaining string. This is only possible if that rule a, b has already been inserted into the grammar. Likewise, in step 3, we need to be able to find the rule ab, ab to be able to find the overlap of the two occurrences of $abab$. The pattern continues, creating larger and larger nodes.



In the example, we see how new nodes constructed from left to right by first recognizing the next node expanding into the right context and then combining it with the previous node. Any next nodes need to interact with the following right context to be of interest, otherwise they would be already represented by the previously read node.

4.3.2 Continuously Updating the Grammar

The algorithm iteratively parses a new node n_i from the input string w and then updates the grammar at the end of each step i , so that after each step, there exists a node y_i representing the prefix of w processed up to step i , so that even those patterns first encountered in the current input string can be recognized by later processing steps.

The general state at step i of the induction algorithm is given by:

- **input string** $w = w_1, \dots, w_n$
- **previous node** p_i constituting the prefix parsed so far
- **next node** n_i starting at token position $x_i \in \{1, \dots, |p_i|\}$ in w

Note that token positions are zero-indexed in this context, which means a starting position of 1 starts after one token has been skipped to only enumerate true postfixes of the previous node.

The next node n_i is combined with the previous node to form the end result node y_i of the i -th step. Depending on the position x_i where n_i has been recognized, the result is constructed in different ways:

1. $x_i = |p_i|$: the new node starts after the previous node and the result node is created by inserting the pattern $[p_i, n_i]$
2. $x_i < |p_i|$: the new node overlaps with the previous node and the result node needs to represent the two nodes in separate partition patterns.

4.3.3 Update Step

In section 3.3 we have established the criterion “Nodes must not have exactly one parent while having no overlapping children”. Therefore, if the previous node is overlapping with the next node while it does not have any other parents, we need to update the previous node to include the expansion from the next node.

If p_i and n_i are overlapping, n_i is an extension of a postfix node $o_i < p_i$. This follows from the fact that all postfixes of p_i have been checked in descending order to find n_i . Thus, we know that p_i and n_i both have a path to their overlap o_i . Also, we know that there is no overlap with p_i that starts before and ends after n_i , because these are iteratively processed in order from largest to smallest postfix in the induction loop.

While updating, it is important to not change the string representation of any nodes which are not meant to be changed. Every node’s string is represented by reconstruction from its child nodes, and we can not change the string of any nodes which have parents outside the range to be updated.

To recognize how to update which nodes, we can build a trace graph for the partition of the overlap and check for any nodes with parents outside this sub-graph. Nodes which are not reachable from outside this sub-graph can be updated directly in-place, while other nodes have to be replaced with a new node to leave the representations of node they occur in unchanged.

Both types of nodes require different updates in a recursive update procedure.

- **in-place updates** can be performed by simply updating all the postfix children to represent the additional tokens
- **re-place updates** are performed by creating a new node containing both the original node and the expansion to represent the expanded range

In-place updates recursively call the update on their child nodes and link to the results of these updates, resulting in a change in their string representation. Since all child nodes are updated to represent the same final string, the node representation remains consistent. While traversing the graph downwards to smaller nodes, eventually encountering re-place nodes becomes more likely. These nodes end the recursion as they can not be changed in place. Instead, the original node and the expansion node are placed into two separate partitions and a new node representing the expanded range is created.

With the update of each individual child node intersected by the update range, the root node of the recursion finally receives a new string representation while the graph structure remains consistent.

5 Discussion

We have now defined the formal structure of the context graph model and have described an algorithm to induce the structure from a given dataset of raw strings. We are now going to discuss some basic principles of how to use the structure as a language model and explore some possible improvements for future applications.

5.1. Predicting Context by Reconstruction

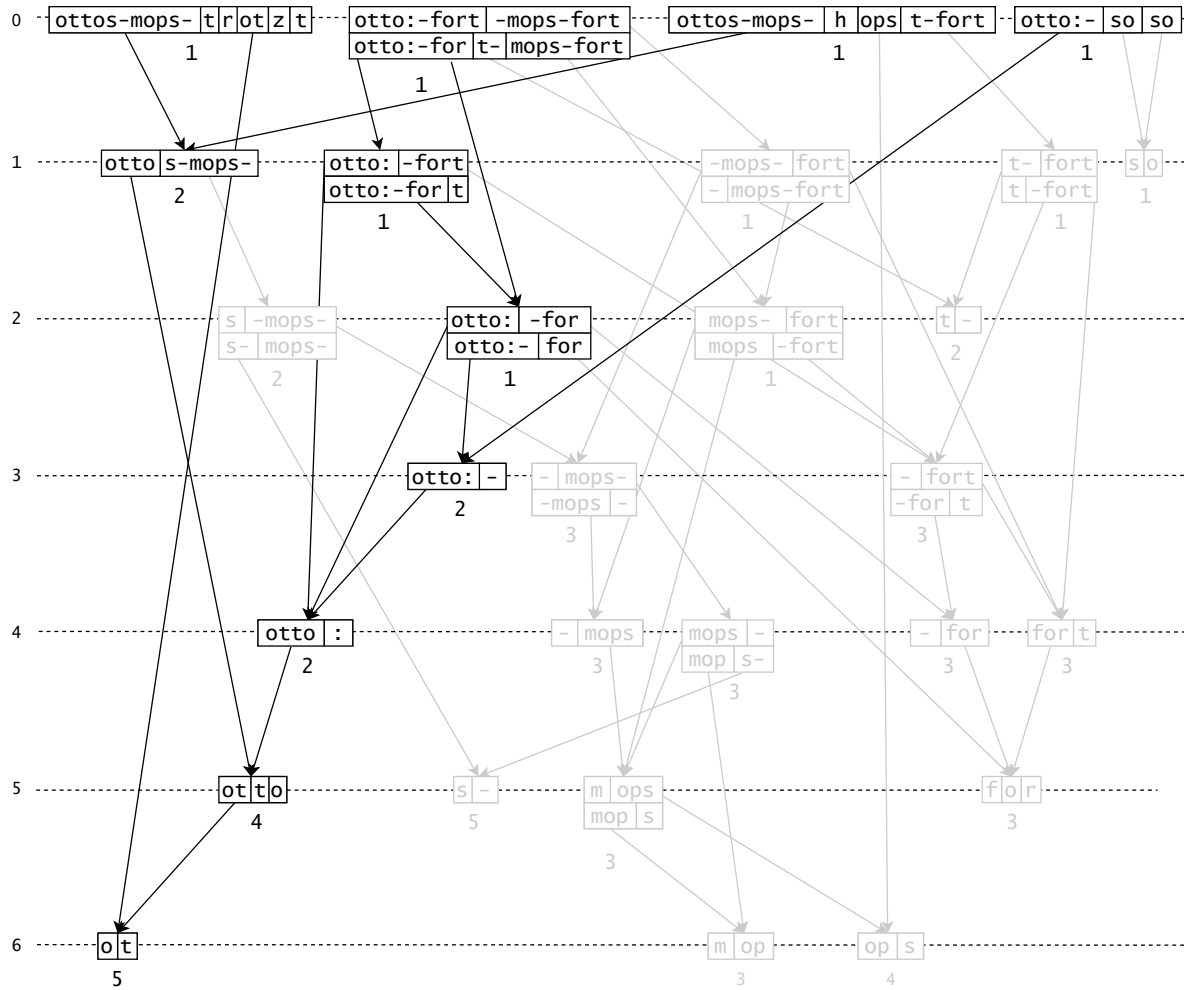


Figure 5.1: All available contexts reachable from the node `ot`.

From the sub-string relation graph we have described, it is easy to model the most likely contexts for a given node in the graph. The conditional probability can be defined identically to regular n-grams.

The probability of a single node can be calculated from the sum of all counts of nodes of the same length n , denoted by the set V_n :

$$P(x \mid v) = \frac{\text{count}(x)}{\text{count}(v)} \qquad P(x) = \frac{\text{count}(x)}{\sum_{v \in V_n} \text{count}(v)}$$

where $x, v \in V$ are nodes representing strings and v is a sub-string of x in G .

Since the formula for n-gram probabilities in definition 2.1 can be applied to arbitrary contexts, the same applies to our model. That relationship holds true not only for direct sub-strings in G , but any pair of sub- and super-strings. Since all the nodes can generally be viewed as n-grams, the prediction of text can generally work the same as in simple n-gram models, although additional extensions are possible.

The key distinction of this structure to n-gram models is that it contains n-grams of all possible lengths from the corpus, while representing them as compressed grammatical rules. The model therefore has a dynamic context window as opposed to classic n-grams which only count a fixed set of n-gram sizes.

Additionally, the edges of the graph make it possible to reconstruct the entire original dataset.

This way the structure can be used as a basic language model. Given an input over the known alphabet of tokens, there exists a node representing at least the first token, and we can reach its various contexts from this node. With each context annotated with a frequency, we can reconstruct the most frequent context that the given prefix is known to occur in.

In the following example the node `ot` has different contexts each of a different frequency. It occurs directly in `ottos-mops-trotzt` and in `otto`. For predicting the most likely completion an algorithm could pick the most frequent of the available contexts to predict the next tokens, the same way as an n-gram model.

$$P(\text{to} \mid \text{ot}) = \frac{4}{5} \qquad P(\text{zt} \mid \text{ot}) = \frac{1}{5}$$

Compared to alternative language modelling approaches, our structure has these specific advantages due to its explicit symbolic structure, efficient representation and dynamic construction procedure, illustrated by the following table:

	N-Grams	Transformers	Context Graph
reconstructable	✗	✗	✓
interpretable	✓	✗	✓
updatable	✓	✗	✓
dynamic context size	✗	✗	✓

Table 5.1: Comparison chart of different language modelling techniques regarding properties discussed in the paper.

5.2. Smoothing Techniques

As with regular n-grams, especially for larger n-grams, data sparsity can be a problem for sensible predictions. As the size of the context grows, the possibilities for different sequences rises exponentially and a training data set will simply not cover all variations of valid phrases. This can also be considered an advantage as it keeps the model within the predefined boundaries it was trained on, however for most applications we want the model to be robust to unseen inputs and be able to handle them appropriately.

One way to allow for responses to unseen inputs of the model is to *smooth* the probability distribution by exploring *similar* n-grams when sampling from the model by some similarity measure. Naive smoothing approaches simply assign all possible n-grams an artificial minimal count to prevent any probabilities to be zero and allow for n-grams which have never been seen in practice to be generated (Laplace Smoothing). These techniques however do not utilize the syntactic or semantic similarity of the n-grams, as would be intuitive, and thus create noise in the output. In practice, we assume the training data to be a subset of a language following some hidden rules and being modified by random noise (such as spelling errors). Thus, we would assume a higher probability for n-grams which are similar to the instances we have actually seen during training.

5.2.1 Syntactic Similarity

Similarity of texts can be defined in terms of syntactic structure or in terms of semantic meaning, which by the distributional hypothesis refers to the similarity of the contexts they occur in.

The syntactic similarity of two texts is usually quantified by their distance in a space over edit operations, also called edit distance. There are several types of edit distance, depending on which edit operations are allowed and how they are counted. We are going to assume the *Levenshtein Distance* for this example.

Definition 5.1 Levenshtein Distance

The Levenshtein Distance $\text{lev}(x_1, x_2)$ of two texts x_1, x_2 is the minimum number of **insertions**, **deletions** or **substitutions** of characters needed to transform one text into the other.

A syntactic similarity metric can then be defined as

$$\text{sim}_{\text{orth}}(x_1, x_2) = 1 - \frac{\text{lev}(x_1, x_2)}{\max\{|x_1|, |x_2|\}}$$

where $\max\{|x_1|, |x_2|\}$ gives the maximum possible Levenshtein distance for x_1 and x_2 . Thus, the similarity ranges between 0 and 1.

A core advantage of the substring structure is the ability to easily traverse the most similar nodes according to the Levenshtein distance. We can derive an upper bound for the distance (or lower bound of syntactic similarity) for the most similar n-grams utilizing the sub-string relation.

Definition 5.2 Edit-Distance in G

For the edit distance $\text{lev}(x_1, x_2) : V \times V \rightarrow \mathbb{N}$ of two nodes $x_1, x_2 \in V$ is

- $\text{lev}(x_1, x_2) = ||x_1| - |x_2||$ if one is a sub-string of the other,
- $\text{lev}(x_1, x_2) \leq \text{lev}(x_1, x_3) + \text{lev}(x_2, x_3)$ if there exists a shared sub-string x_3
- $\text{lev}(x_1, x_2) = \max\{|x_1|, |x_2|\}$ if there are no shared sub-strings

Because each node is connected to its largest sub-strings and smallest super-strings, we can traverse the set of syntactically similar strings in a similarity-descending order by visiting the super-strings of sub-strings. With each edge traversed we either add to or remove a minimal amount of context from a given node and explore neighborhoods of the largest shared substrings first.

5.2.2 Semantic Similarity

The semantic similarity of two texts can be defined on lexical terms as the similarity of the possible contexts the texts may appear in. Thus, the semantic distance of two distinct n-grams can be defined as the average syntactic edit-distance between all of their contexts:

Definition 5.3 Semantic Distance

The semantic distance $\text{dist}_{\text{sem}}(x_1, x_2) : V \times V \rightarrow \mathbb{N}$ of two nodes $x_1, x_2 \in V$ is the average minimum edit distance between contexts of x_1 and x_2

$$\text{dist}_{\text{sem}}(x_1, x_2) := \text{avg}(\{\min\{\text{lev}(c_1, c_2) \mid c_2 \in \text{Ctx}(x_2)\} \mid c_1 \in \text{Ctx}(x_1)\})$$

The semantic distance captures the syntactic edit operations required to transform the known contexts of one text into the contexts of the other text and thus quantifies their degree of compatibility with the remaining corpus.

Similarly to how attention weights in a self-attention unit of the transformer architecture are trained to point to their contexts in a hyper-dimensional latent space, the nodes in a sub-string relation are linked with other strings they appear in context with. By comparing the similarity of these links we can quantify the semantic similarity by the same definition of distributional semantics.

One difference is that in the self-attention mechanism, tokens link to other tokens, even when they never appear in the same context during training. When a and b appear in the same context and thus a is weighed towards b , a is also weighed towards tokens similar to b even if they have never been seen in the same contexts.

To replicate this behaviour in the sub-string relation graph, we can modify the edit distance of contexts to discount replacements with high semantic similarity. The semantic distance between two texts should be lower when the contexts can be transformed into one another using semantically similar nodes.

To find the semantically most similar nodes to a given node, we find the possible replacements for it in its contexts. This can be achieved by finding the alternative contexts for each context node of the original node. In a recursive procedure we keep traversing the sub-string relation in a breadth-first manner until we find nodes with no contexts or no children, keeping track of the minimum path lengths for each transformation. Because the structure stores direct edges between the most similar sub-strings, paths between most similar nodes will be shortest and a lower bound for similarity can be placed on transformations not fully explored. This way the semantic similarity of nodes can be incrementally approximated according to resource requirements.

5.3. Accepting Novel Inputs

For inputs seen in the training corpus it is easy to predict the appropriate context, as there exists a node in the graph structure to derive contexts from. However, in general, input strings can be composed of novel combinations of tokens which have only partially been seen in training, if at all. To allow the model to predict context for unknown sequences, we need to allow it to draw information from nodes similar to the input.

A simple way to address this in common n-gram models is to use *backoff smoothing*.

Definition 5.4 Backoff Smoothing

The Backoff probability of an n-gram $w_{i-n+1}, \dots, w_i$ is defined as

$$P_{bo}(w_i \mid w_{i-n+1}, \dots, w_{i-1}) = \begin{cases} \frac{C(w_{i-n+1}, \dots, w_i)}{C(w_{i-n+1}, \dots, w_{i-1})} & \text{if } C(w_{i-n+1}, \dots, w_i) > 0, \\ P_{bo}(w_i \mid w_{i-n+2}, \dots, w_{i-1}) & \text{otherwise} \end{cases}$$

Meaning the length of context to be conditioned upon will be reduced as long as there exists no support for the resulting n-gram in the model. Instead, the n-gram probability of the largest context available in the model is used.

We could apply the same form of smoothing in the hierarchy of the sub-string relation, but it would certainly be better to search for nodes syntactically or semantically similar to the input and use their contexts for prediction instead.

6 Conclusion

In this paper, we motivated the need for reliable, interpretable and updatable language models. We have defined a formal structure based on a substring relation graph able to relate all frequent substrings of a text corpus to their contexts and have shown how this structure can be used to predict following tokens of a given input. Further we have shown that the original dataset is completely reconstructable from the derived structure and that the rules are human-readable and interpretable. We then described a theoretical basis for an induction algorithm learning this structure from a data set of example strings, where we have shown that the rules of the model are directly updatable without corrupting previously learned data. Finally, we discussed examples of how to apply the context graph as a language model and explored possible future extensions to the model using its internal structure to make it more robust.

In conclusion, we have presented a novel framework for inducing context relationships between repetitive sequential patterns with high relevance to standing challenges in the field of natural language processing. From this theoretical basis we hope to build an implementation of this model and test it in a real-world setting. We see many possible extensions to this framework to increase the comprehensive abilities of this language model and make it more comparable to neural network based approaches. In a realistic setting, we can see the current model play an auxiliary role for analyzing datasets and deriving hyper-dimensional embeddings from nodes in the graph or inducing more formal relationships between nodes in the graph. We are also interested in applying this model in other contexts, for example in search engines or hierarchical action planning.

Bibliography

- [1] Joseph Weizenbaum. “ELIZA—a computer program for the study of natural language communication between man and machine”. In: *Communications of the ACM* 9.1 (1966), pp. 36–45.
- [2] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.
- [3] Ziwei Ji et al. “Survey of hallucination in natural language generation”. In: *ACM Computing Surveys* 55.12 (2023), pp. 1–38.
- [4] Katja Filippova. “Controlled hallucinations: Learning to generate faithfully from noisy data”. In: *arXiv preprint arXiv:2010.05873* (2020).
- [5] Oana-Maria Camburu et al. “Make up your mind! adversarial generation of inconsistent natural language explanations”. In: *arXiv preprint arXiv:1910.03065* (2019).
- [6] Tobin South, Robert Mahari, and Alex Pentland. “Transparency by Design for Large Language Models”. In: *Computational Legal Futures, Network Law Review*.(2023) (2023).
- [7] Minhyeok Lee. “A Mathematical Investigation of Hallucination and Creativity in GPT Models”. In: *Mathematics* 11.10 (2023). ISSN: 2227-7390. DOI: 10 . 3390 / math11102320. URL: [https : //www .mdpi .com/2227-7390/11/10/2320](https://www.mdpi.com/2227-7390/11/10/2320).
- [8] Niv Haim et al. *Reconstructing Training Data from Trained Neural Networks*. 2022. arXiv: 2206 . 07758 [cs.LG].
- [9] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive nlp tasks”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 9459–9474.
- [10] Fabio Petroni et al. “Language models as knowledge bases?” In: *arXiv preprint arXiv:1909.01066* (2019).
- [11] Paul F Christiano et al. “Deep reinforcement learning from human preferences”. In: *Advances in neural information processing systems* 30 (2017).
- [12] Constituency Parsing. “Speech and language processing”. In: *Power Point Slides* (2009).
- [13] Tomas Mikolov et al. “Efficient estimation of word representations in vector space”. In: *arXiv preprint arXiv:1301.3781* (2013).
- [14] Ashish Vaswani et al. “Attention Is All You Need”. In: *CoRR* abs/1706.03762 (2017). arXiv: 1706 . 03762. URL: <http://arxiv.org/abs/1706.03762>.
- [15] Craig G Nevill-Manning and Ian H Witten. “Identifying hierarchical structure in sequences: A linear-time algorithm”. In: *Journal of Artificial Intelligence Research* 7 (1997), pp. 67–82.
- [16] Vilém Zouhar et al. “A formal perspective on byte-pair encoding”. In: *arXiv preprint arXiv:2306.16837* (2023).
- [17] Greg Orosi. “A simple derivation of Faulhaber’s formula”. In: *Appl. Math. E-Notes* 18 (2018), pp. 124–126.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Bachelorarbeit selbstständig verfasst und gelieferte Datensätze, Zeichnungen, Skizzen und graphische Darstellungen selbstständig erstellt habe. Ich habe keine anderen Quellen als die angegebenen benutzt und habe die Stellen der Arbeit, die anderen Werken entnommen sind - einschließlich verwendeter Tabellen und Abbildungen - in jedem Fall unter Angabe der Quelle als Entlehnung kenntlich gemacht.

Name: Linus Behrbohm

Bielefeld, 11. Dezember 2023

Datum

Unterschrift